## Plot from a DataFrame

Plot graphs directly using dataframes. Plot function using matplotlib in the backend and offer simiplicity in usage.

Scatter diagram

1. *Line chart*
2. *Bar chart*
3. *Pie chart*

## Plot a Scatter Diagram using Pandas

Scatter plots are used to depict a relationship between two variables. Here are the steps to plot a scatter diagram using Pandas.

**Step 1: Prepare the data**

To start, prepare the data for your scatter diagram.

**Step 2: Create the DataFrame**

Once you have your data ready, you can proceed to create

the DataFrame in Python.

```
import pandas as pd
data = {'Unemployment_Rate': [6.1, 5.8, 5.7, 5.7, 5.8, 5.6, 5.5, 5.3, 5.2, 5.2],
        'Stock_Index_Price': [1500, 1520, 1525, 1523, 1515, 1540, 1545, 1560, 1555, 1565]
        }
```

```
df = pd.DataFrame(data, columns=['Unemployment_Rate',
'Stock_Index_Price'])
print(df)
```

*output:*

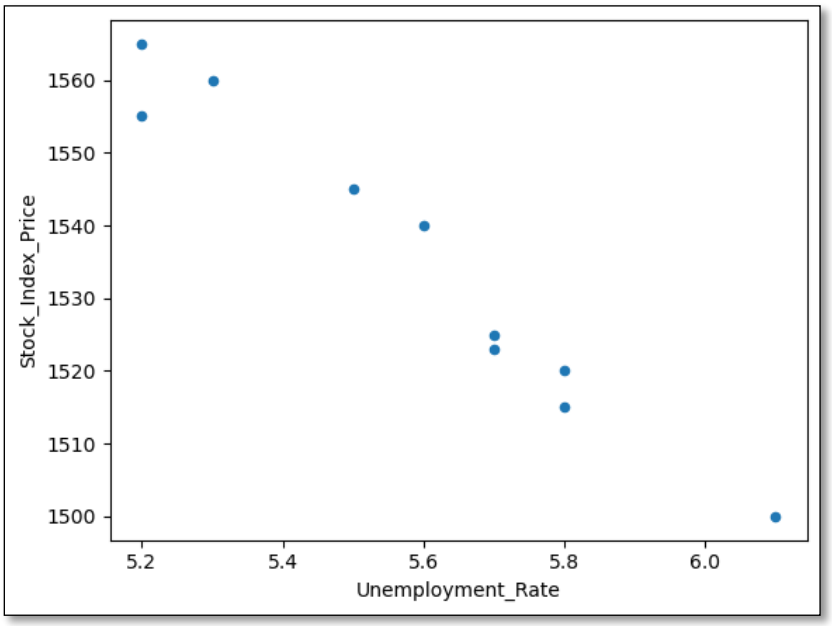
| | Unemployment_Rate | Stock_Index_Price |
|---|---|---|
| 0 | 6.1 | 1500 |
| 1 | 5.8 | 1520 |
| 2 | 5.7 | 1525 |
| 3 | 5.7 | 1523 |
| 4 | 5.8 | 1515 |
| 5 | 5.6 | 1540 |
| 6 | 5.5 | 1545 |
| 7 | 5.3 | 1560 |
| 8 | 5.2 | 1555 |
| 9 | 5.2 | 1565 |

### Step 3: Plot the DataFrame using Pandas

Finally, you can plot the DataFrame by adding the following syntax:

df.plot(x ='Unemployment_Rate', y='Stock_Index_Price', kind = 'scatter')

```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Unemployment_Rate': [6.1, 5.8, 5.7, 5.7, 5.8, 5.6, 5.5, 5.3, 5.2, 5.2],
        'Stock_Index_Price': [1500, 1520, 1525, 1523, 1515, 1540, 1545, 1560, 1555, 1565]
        }
df = pd.DataFrame(data, columns=['Unemployment_Rate', 'Stock_Index_Price'])
df.plot(x='Unemployment_Rate', y='Stock_Index_Price', kind='scatter')
plt.show()
```




## Plot a Line Chart using Pandas

Line charts are often used to display trends overtime.

Let’s now see the

steps to plot a line chart using Pandas.

**Step 1: Prepare the data**

To start, prepare your data for the line chart.

**Step 2: Create the DataFrame**

**Step 3: Plot the DataFrame using Pandas**

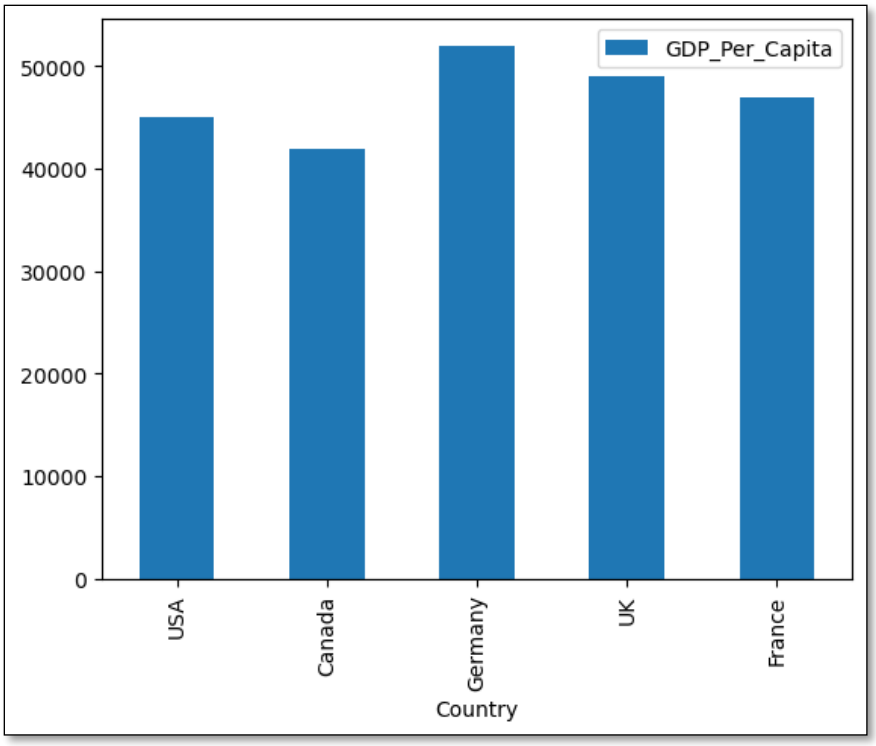
```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year':
[1920,1930,1940,1950,1960,1970,1980,1990,2000,2010],
        'Unemployment_Rate':
[9.8,12,8,7.2,6.9,7,6.5,6.2,5.5,6.3]
       }
df = pd.DataFrame(data,columns=['Year','Unemployment_Rate'])
df.plot(x ='Year', y='Unemployment_Rate', kind = 'line')
plt.show()
```




## Plot a Bar Chart using Pandas

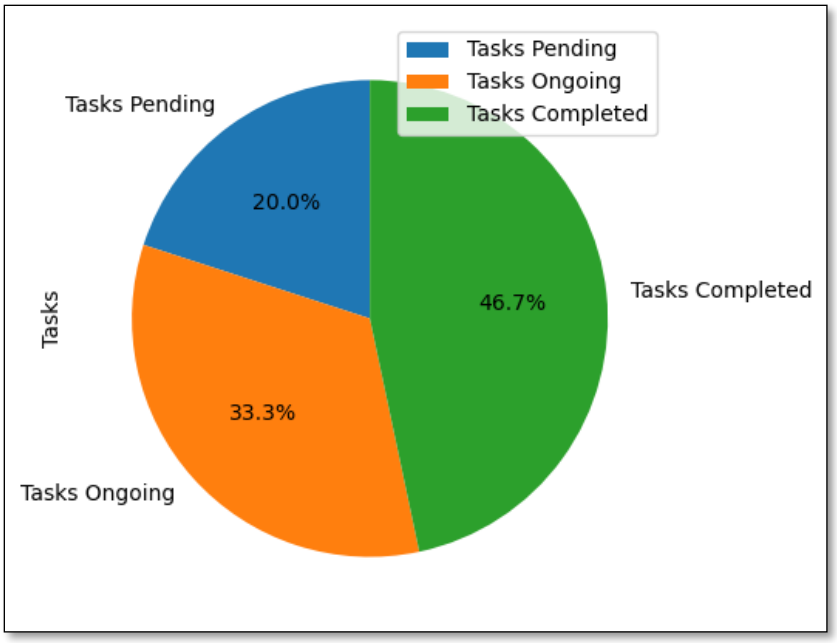
Bar charts are used to display categorical data. Let's now see how to plot a bar chart using Pandas.

```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Country': ['USA','Canada','Germany','UK','France'],
        'GDP_Per_Capita': [45000,42000,52000,49000,47000]
       }
df = pd.DataFrame(data,columns=['Country','GDP_Per_Capita'])
df.plot(x ='Country', y='GDP_Per_Capita', kind = 'bar')
plt.show()
```




## Plot a Pie Chart using Pandas

```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Tasks': [300,500,700]}
df = pd.DataFrame(data,columns=['Tasks'],index = ['Tasks
Pending','Tasks Ongoing','Tasks Completed'])
df.plot.pie(y='Tasks',figsize=(5, 5),autopct='%1.1f%%',
startangle=90)
plt.show()
```




## Time Series

Pandas provide a robust tool for working time with Time series data, especially in the financial sector. While working with time series data, we frequently come across the following −

Generating sequence of time

Convert the time series to different frequencies

Get Current Time

datetime.now() gives you the current date and time.

```
import pandas as pd
print(pd.datetime.now())
```

*output:*

*2022-11-25 16:13:00.012918*

## Create a TimeStamp

Time-stamped data is the most basic type of timeseries data that associates values with points in time. For pandas objects, it means using the points in time.

```
import pandas as pd
print(pd.Timestamp('2017-03-01'))
```

*output:*

*2017-03-01 00:00:00*

### Create a Range of Time

```
import pandas as pd
print(pd.date_range("11:00", "13:30", freq="30min").time)
```

*output:*

*[datetime.time(11, 0) datetime.time(11, 30) datetime.time(12, 0)*

*datetime.time(12, 30) datetime.time(13, 0) datetime.time(13, 30)]*

## Change the Frequency of Time

```
import pandas as pd
print(pd.date_range("10:00", "13:30", freq="H").time)
```

*output:*

*[datetime.time(10, 0) datetime.time(11, 0) datetime.time(12, 0)*

*datetime.time(13, 0)]*

## Date Functionality

Extending the Time series, Date functionalities play major role in financial data analysis. While working with Date data, we will frequently come across the following −

Generating sequence of dates

Convert the date series to different frequencies

## Create a Range of Dates

Using the date_range() function by specifying the periods and the frequency, we can create the date series. By default, the frequency of range is Days.

```
import pandas as pd
```

```
print(pd.date_range('1/1/2011', periods=5))
```

*output:*

*DatetimeIndex(['2011-01-01', '2011-01-02', '2011-01-03', '2011-01-04',*

*'2011-01-05'],*

*dtype='datetime64[ns]', freq='D')*

## Change the Date Frequency

freq= 'D' for daily

1. *'D' = daily*
2. *'H' = hourly*
3. *'M' = month end*
4. *'B' = business day*
5. *'W' = weekly*
6. *'Q' = quarter end*

```
import pandas as pd
print(pd.date_range('1/1/2011', periods=5,freq='M'))
```

*output:*

*DatetimeIndex(['2011-01-31', '2011-02-28', '2011-03-31', '2011-04-30',*

*'2011-05-31'],*

*dtype='datetime64[ns]', freq='M')*

### Create date range from start to end

```
import pandas as pd
start = pd.datetime(2011, 1, 1)
end = pd.datetime(2011, 1, 5)
print(pd.date_range(start, end))
```

*output:*

*DatetimeIndex(['2011-01-01', '2011-01-02', '2011-01-03', '2011-01-04',*

*'2011-01-05'],*

*dtype='datetime64[ns]', freq='D')*

### Create a DataFrame with date values

```
import pandas as pd

# Create a date range
date_rng = pd.date_range(start='2024-01-01', end='2024-01-
10', freq='D')

# Create DataFrame with 'date' as a column and a
corresponding 'data' column
df = pd.DataFrame({
    'date': date_rng,
    'data': range(len(date_rng))
})
```

```
# Print the DataFrame
print(df)
```

## date

pd.Series.dt - Accessor for DateTime Methods

The dt accessor provides a lot of functionality for working with datetime data.

```
import pandas as pd
# Sample data
dates = pd.Series(pd.date_range('2025-01-01', periods=5,
freq='D'))
```

## Extract day of the week

```
day_of_week = dates.dt.day_name()
print(day_of_week)
```

*output:*

*0 Wednesday*

*1 Thursday*

*2 Friday*

*3 Saturday*

*4 Sunday*

*dtype: object*

**Extract month of the year**

```
import pandas as pd
# Sample data
dates = pd.Series(pd.date_range('2025-01-01', periods=5,
freq='M'))

# Extract day of the week
day_of_month = dates.dt.month_name()
print(day_of_month)
```

*Output:*

*0 January*

*1 February*

*2 March*

*3 April*

*4 May*

*dtype: object*

```
#Extract Business days, Excludes Saturday and Sundays
```

```
import pandas as pd
# Sample data
dates = pd.Series(pd.date_range('2025-07-01', periods=5,
freq='B'))

print(dates)
```

*output:*

```
0   2025-07-01
1   2025-07-02
2   2025-07-03
3   2025-07-04
4   2025-07-07
dtype: datetime64[ns]
```

### Filter a DataFrame Between Two Dates

```
import pandas as pd

# Create a date range
date_rng = pd.date_range(start='2024-01-01', end='2024-01-
10', freq='D')

# Create DataFrame with 'date' as a column and a
corresponding 'data' column
df = pd.DataFrame({
```

```
    'date': date_rng,
    'data': range(len(date_rng))
})


df['date'] = pd.to_datetime(df['date']) # convert to date
format

# Filter using a boolean mask
mask = (df['date'] >= '2024-01-03') & (df['date'] <= '2024-
01-07')
filtered_df = df[mask]
print(filtered_df)
```

*output:*

```
        date  data
2 2024-01-03     2
3 2024-01-04     3
4 2024-01-05     4
5 2024-01-06     5
6 2024-01-07     6
```

## Timedelta

Timedeltas are differences in times, expressed in difference units, for example, days, hours, minutes, seconds. They can be both positive and negative. We can create Timedelta objects using various arguments as shown below −

### String

By passing a string literal, we can create a timedelta object.

```
import pandas as pd
print(pd.Timedelta('2 days 2 hours 15 minutes 30 seconds'))
```

*output:*

*2 days 02:15:30*

### Add a Timedelta to a Date Column

```
import pandas as pd

# Create a DataFrame with a datetime column
df = pd.DataFrame({
    'event_time': pd.date_range('2024-01-01 08:00:00',
periods=5, freq='D')
})

# Define a timedelta
delta = pd.Timedelta('2 days 2 hours 15 minutes 30 seconds')

# Create a new column with the adjusted time
df['event_end'] = df['event_time'] + delta
```

```
print(df)
```

*output:*

```
           event_time           event_end
0 2024-01-01 08:00:00 2024-01-03 10:15:30
1 2024-01-02 08:00:00 2024-01-04 10:15:30
2 2024-01-03 08:00:00 2024-01-05 10:15:30
3 2024-01-04 08:00:00 2024-01-06 10:15:30
4 2024-01-05 08:00:00 2024-01-07 10:15:30
```

### Combine Duration Columns into a Timedelta

```
import pandas as pd
import numpy as np

# Create random duration components
np.random.seed(42)
df = pd.DataFrame({
    'days': np.random.randint(0, 5, size=10),
    'hours': np.random.randint(0, 24, size=10),
    'minutes': np.random.randint(0, 60, size=10),
    'seconds': np.random.randint(0, 60, size=10)
})

# Combine into a Timedelta column
df['duration'] = pd.to_timedelta(
    df['days'].astype(str) + ' days ' +
    df['hours'].astype(str) + ' hours ' +
    df['minutes'].astype(str) + ' minutes ' +
```

```
    df['seconds'].astype(str) + ' seconds'
)

print("Full DataFrame:\n", df)
```

*output:*

```
Full DataFrame:
    days  hours  minutes  seconds        duration
0     3      3        1       26 3 days 03:01:26
1     4      7       59       58 4 days 07:59:58
2     2     23       20       41 2 days 23:20:41
3     4      2       32       27 4 days 02:32:27
4     4     21       11       59 4 days 21:11:59
5     1     20       57       15 1 days 20:57:15
6     2      1       21       14 2 days 01:21:14
7     2     23       43       46 2 days 23:43:46
8     2     11       24       50 2 days 11:24:50
9     4      5       48       43 4 days 05:48:43
```

**Integer**

By passing an integer value with the unit, an argument creates a Timedelta object.

```
import pandas as pd
print(pd.Timedelta(6,unit='h'))
```

*output:*

*0 days 06:00:00*

## Data Offsets

Data offsets such as - weeks, days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds can also be used in construction.

```
import pandas as pd
print(pd.Timedelta(days=2))
```

*output:*

*2 days 00:00:00*

## Operations

You can operate on Series/ DataFrames and construct timedelta64[ns] Series through subtraction operations on datetime64[ns] Series, or Timestamps.

```
import pandas as pd
s = pd.Series(pd.date_range('2012-1-1', periods=3,
freq='D'))
```

```
td = pd.Series([ pd.Timedelta(days=i) for i in range(3) ])
df = pd.DataFrame(dict(A = s, B = td))
print(df)
```

*output:*

*A B*

*0 2012-01-01 0 days*

*1 2012-01-02 1 days*

*2 2012-01-03 2 days*

## Filter data based on Timedelta

Perform filtering on column based on conditions

**Example 1: Filter data greater than 2 days 6 hours**

```
import pandas as pd
import numpy as np

# Create random duration components
np.random.seed(42)
df = pd.DataFrame({
    'days': np.random.randint(0, 5, size=10),
    'hours': np.random.randint(0, 24, size=10),
    'minutes': np.random.randint(0, 60, size=10),
    'seconds': np.random.randint(0, 60, size=10)
})
```

```
# Combine into a Timedelta column
df['duration'] = pd.to_timedelta(
    df['days'].astype(str) + ' days ' +
    df['hours'].astype(str) + ' hours ' +
    df['minutes'].astype(str) + ' minutes ' +
    df['seconds'].astype(str) + ' seconds'
)

filtered_df = df[df['duration'] > pd.Timedelta('2 days 6
hours')]
print("\nFiltered > 2 days 6 hours:\n", filtered_df)
```

**Example 2: Filter between 1 and 3 days**

```
import pandas as pd
import numpy as np

# Create random duration components
np.random.seed(42)
df = pd.DataFrame({
    'days': np.random.randint(0, 5, size=10),
    'hours': np.random.randint(0, 24, size=10),
    'minutes': np.random.randint(0, 60, size=10),
    'seconds': np.random.randint(0, 60, size=10)
})
```

```
# Combine into a Timedelta column
df['duration'] = pd.to_timedelta(
    df['days'].astype(str) + ' days ' +
    df['hours'].astype(str) + ' hours ' +
    df['minutes'].astype(str) + ' minutes ' +
    df['seconds'].astype(str) + ' seconds'
)
filtered_range = df[(df['duration'] >= pd.Timedelta('1
days')) & (df['duration'] <= pd.Timedelta('3 days'))]
print("\nFiltered between 1 and 3 days:\n", filtered_range)
```

### Addition Operations

```
import pandas as pd
s = pd.Series(pd.date_range('2012-1-1', periods=3,
freq='D'))
td = pd.Series([ pd.Timedelta(days=i) for i in range(3) ])
df = pd.DataFrame(dict(A = s, B = td))
df['C']=df['A']+df['B']
print(df)
```

*output:*

*A B C*

*0 2012-01-01 0 days 2012-01-01*

*1 2012-01-02 1 days 2012-01-03*

*2 2012-01-03 2 days 2012-01-05*

### Subtraction Operation

```
import pandas as pd
s = pd.Series(pd.date_range('2012-1-1', periods=3,
freq='D'))
td = pd.Series([ pd.Timedelta(days=i) for i in range(3) ])
df = pd.DataFrame(dict(A = s, B = td))
df['C']=df['A']-df['B']
df['D']=df['C']-df['B']

print(df)
```

*output:*

*A B C D*

*0 2012-01-01 0 days 2012-01-01 2012-01-01*

*1 2012-01-02 1 days 2012-01-01 2011-12-31*

*2 2012-01-03 2 days 2012-01-01 2011-12-30*

### Exporting dataframe

Extracting the cleaned data to csv, excel for future analysis is important in data analytics.

#### 1. Saving a DataFrame to a CSV File: df.to_csv()

To save a Pandas DataFrame to a CSV file, you can use the to_csv() method. Here's an example:

```
import pandas as pd
# Sample DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': ['a', 'b', 'c']
})
# Save the DataFrame to a CSV file
df.to_csv('data.csv', index=False)  # index=False prevents
saving the index to the file
print("DataFrame saved as CSV")
```

### 2. Saving a DataFrame to an Excel File: df.to_excel()

To save a Pandas DataFrame to an Excel file, you can use the to_excel() method. You need to have the openpyxl or xlsx writer library installed to write to Excel files.

```
import pandas as pd
# Sample DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': ['a', 'b', 'c']
})
```

```
# Save the DataFrame to an Excel file
df.to_excel('data.xlsx', index=False)
print("DataFrame saved as Excel file")
```

### 3. Saving Multiple DataFrames to Multiple Sheets in Excel: ExcelWriter

If you want to save multiple DataFrames to different sheets in a single Excel file, you can use the ExcelWriter context manager.

```
import pandas as pd
# Sample DataFrames
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'C': [7, 8, 9], 'D': [10, 11, 12]})
# Save both DataFrames to different sheets in one Excel file
with pd.ExcelWriter('multi_sheets.xlsx', engine='openpyxl')
as writer:
    df1.to_excel(writer, sheet_name='Sheet1', index=False)
    df2.to_excel(writer, sheet_name='Sheet2', index=False)
print("Multiple DataFrames saved to Excel file with multiple
sheets")
```

### 4. Saving DataFrame to JSON:

```
import pandas as pd
# Sample DataFrame
df = pd.DataFrame({
```

```
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': ['a', 'b', 'c']
})
# Save the DataFrame to a JSON file
df.to_json('data.json', orient='records', lines=True)
print("DataFrame saved as JSON")
```